

Autonomous Vehicle System

Hussein Mohamud, Aidan Elkins, Emily Bailey, William Varda, Rami
Ibraheem

System Description:

This system handles car rentals and overall day-to-day management for BeAvis. It has the option of being accessible via both website and mobile application (with latest IOS or Android version), each with the same utilities. Users can enable GPS to view nearby locations as well as their direction once a rental is requested.

Users must create an account with their email and a password to make rentals. After verifying an email, their account will be created and they will then have the option to enable two-factor authentication with a separate authenticator app at any time, although it is not mandatory. Users can make rentals, view rental history, find locations, and link their method of payment. Payment information is secure and in a different location from other data in case of breach.

Rental agreement contracts are also given and signed here; the contracts will be kept secure along with customer signatures.

Employees can log into the system and are able to access customers' rental status and contracts, see cars available for rental, and update car status.

Software Architecture Overview:

This is placeholder text... talk about or show Architectural Diagram of all major components

Here, either show or mention the UML class diagram if it is being sent as a separate file

Description of classes:

User:

Description of Classes, Associated Attributes & Operations

Class name: Car Database

Purpose: To store and update all stored care information

Parameters

+ databaseNumer: int

+ totalCars: int

+ location: int

+ carList: List<Car>

No Methods

Class name: Car

Purpose: To create and update all stored car objects

Parameters

+ brand: String

+ model: String

+ Condition: String

+ vehicleID: int

+ available: boolean

+ location: int

+ mileage: int

+ cost: float

Methods

+ changeCondition(condition): void

updates stored car condition

+ changeAvailable(available): void

updates stored car availability

+ changeLocation(location): void

updates stored car location

+ deleteCar(): void

removes car from database

Class name: User

Purpose: Abstract class for both user types

Parameters

+ email: String

- password: String

No Methods

Development plan and timeline

Partitioning of tasks:

Rami: Documentation & Software Summary - assisted UML

Hussein: Primary UML

Aidan: Software Architectural Diagram - assisted UML

William: Primary UML

Emily: Class and Method Descriptions - assisted UML

Team member responsibilities:

Submission of final project: Emily

Verify Git Organization: Aiden